

Jump Game 2

Problem Statement: You are given a 0-indexed array `nums` of length `n` representing your maximum jump capability from each index.

- You start at index 0. Each element `nums[i]` represents the maximum number of steps you can jump forward from index `i`.
- Your goal is to reach the last index of the array (`nums[n - 1]`) using the minimum number of jumps
- Return the minimum number of jumps required to reach the last index.

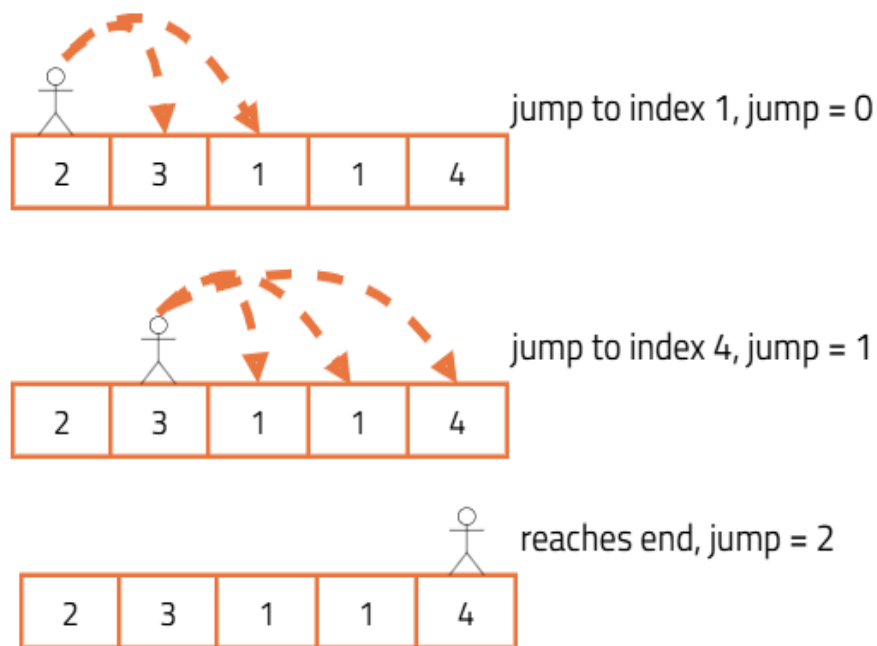
You can assume that it is always possible to reach the last index.

Examples

Input: `nums = [2, 3, 1, 1, 4]`

Output: 2

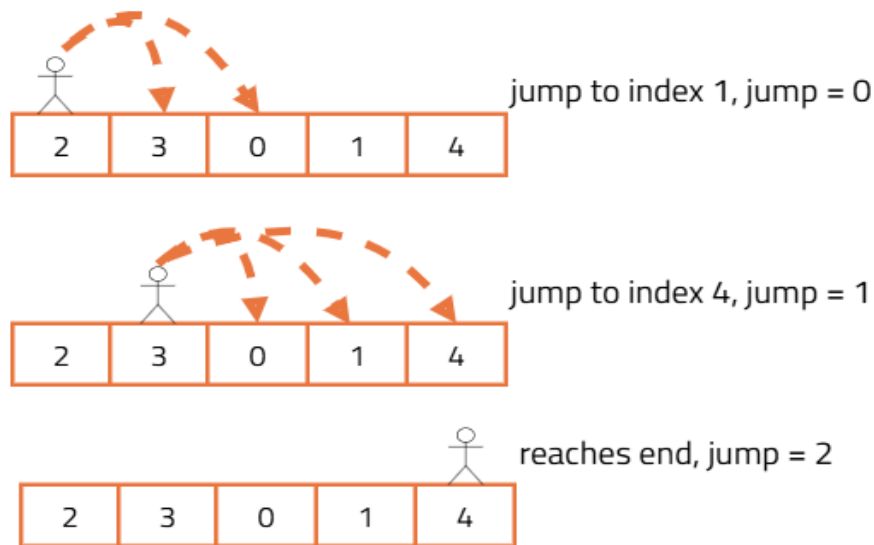
Explanation: Jump from index 0 → 1 → 4.



Input: `nums = [2, 3, 0, 1, 4]`

Output: 2

Explanation: Jump from index 0 → 1 → 4.



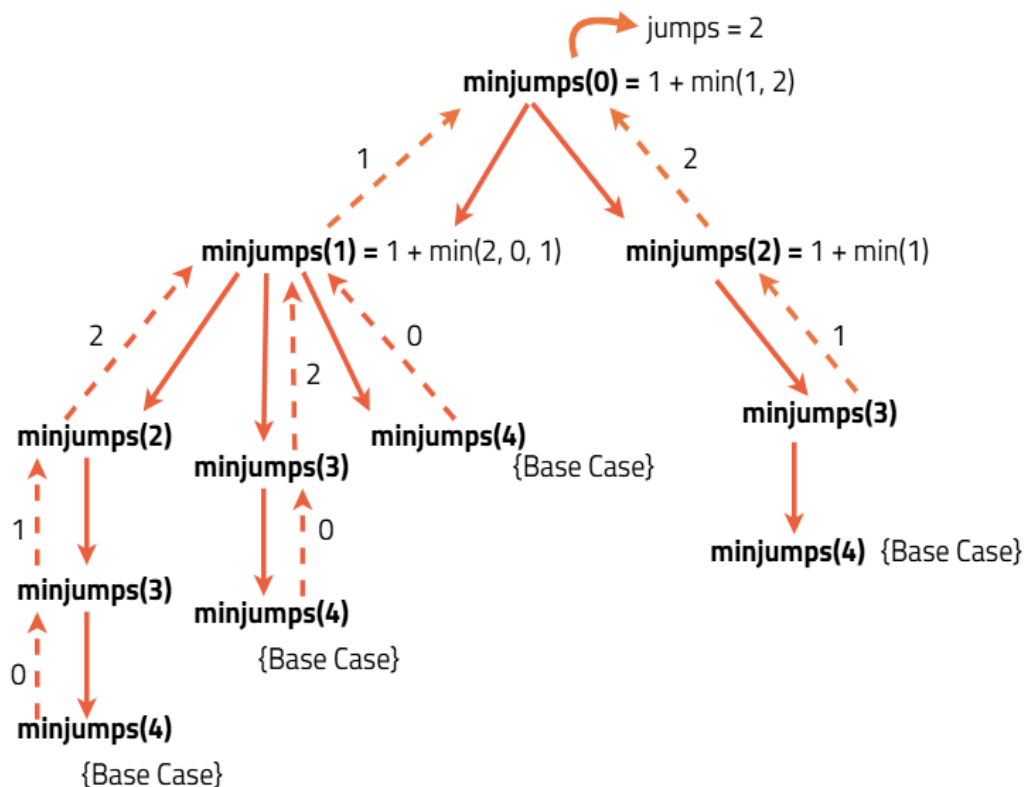
Brute force Approach

Algorithm

The brute force idea is to try every possible jump from each index. From the starting position, we explore all jumps within the allowed range `nums[i]`. For each position we land on, we again try all possible jumps from there. This process continues until we either reach the end or exhaust all paths. We recursively calculate the minimum number of jumps needed to reach the end from each index. Since we're trying all combinations, it's not efficient but ensures we check every possible path.

1. Define a recursive function that takes the current index as input.
2. If the current index is at or beyond the last index, return 0 (we're already at the end).
3. Initialize a variable to store the minimum number of jumps (e.g., `minJumps = INF`).
4. Loop through all possible jumps from current index (from 1 to `nums[i]`).
5. For each jump, call the recursive function for the new index (`i + jump`).
6. Update `minJumps` with the minimum of its current value and `1 + result of recursive call`.
7. After exploring all paths, return `minJumps`.

Dry run:



Code

CppJavaPythonJavascript

```
// Class to define the solution logic
class Solution {
    // Main jump function
    public int jump(int[] nums) {
        return minJumps(nums, 0);
    }

    // Helper method to recursively calculate the minimum number of jumps
    private int minJumps(int[] nums, int position) {
        // If current position is at or beyond the last index
        if (position >= nums.length - 1) return 0;

        // If stuck at a 0
        if (nums[position] == 0) return Integer.MAX_VALUE;

        int minStep = Integer.MAX_VALUE;

        // Try every jump from 1 to nums[position]
        for (int jump = 1; jump <= nums[position]; jump++) {
            int subResult = minJumps(nums, position + jump);
            if (subResult != Integer.MAX_VALUE) {
                minStep = Math.min(minStep, 1 + subResult);
            }
        }

        return minStep;
    }
}
```

```

}

// Driver class with main method
public class JumpGame {
    public static void main(String[] args) {
        int[] nums = {2, 3, 1, 1, 4};

        Solution sol = new Solution();
        System.out.println("Minimum number of jumps: " + sol.jump(nums));
    }
}

```

Complexity Analysis

Time Complexity: $O(2^N)$, where N is the number of elements in the array. This is because, from each index, the function recursively explores all possible jump lengths, leading to an exponential number of recursive calls.

Space Complexity: $O(N)$, due to the maximum depth of the recursion stack in the worst case. No extra data structures are used except the recursive call stack.

Better Approach

Algorithm

We use dynamic programming to track the minimum number of jumps needed to reach each index. We initialize a dp array where each $dp[i]$ holds the minimum jumps required to reach index i . Set $dp[0] = 0$ (starting point), and all other entries as infinity. For every index, explore how far you can jump and update the dp value for each reachable position. The value at $dp[n - 1]$ will be our answer.

- Initialize a list of size equal to the input array length with very large values.
- Set the first index value to 0 since no jumps are needed at the start.
- Iterate through the array from start to end.
- From each index, check how far you can jump based on the value at that index.
- For each reachable index from the current position, update it with the minimum jumps required.
- Return the value at the last index of the list, which represents the minimum jumps needed to reach the end.

Dry run:

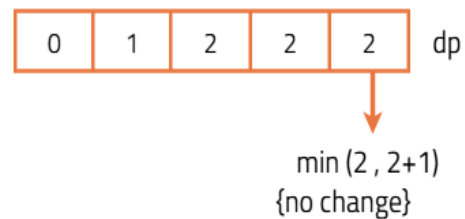
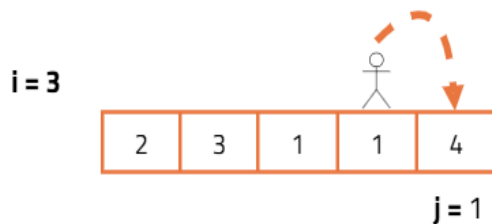
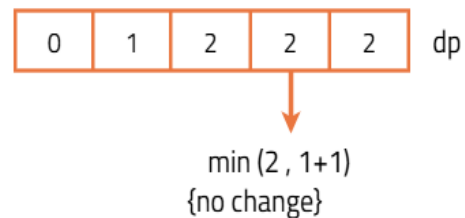
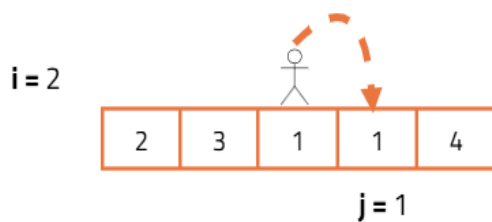
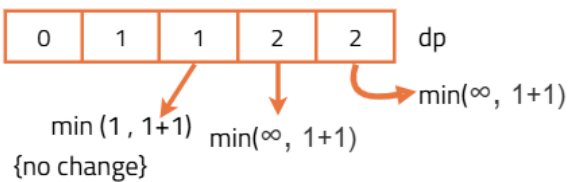
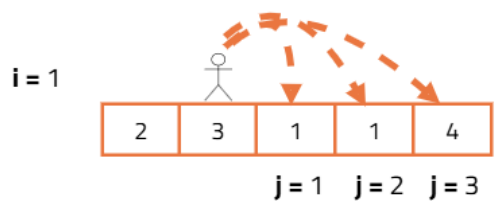
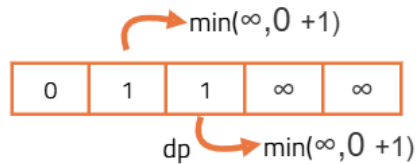
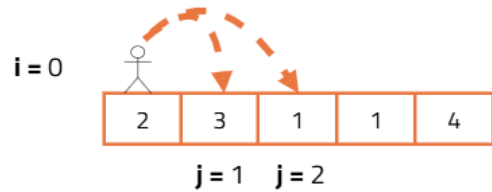
nums =

2	3	1	1	4
---	---	---	---	---

dp =

∞	∞	∞	∞	∞
----------	----------	----------	----------	----------

$$dp[i + j] = \min(dp[i+j], dp[i] + 1)$$



no further jumps dp[4] = 2

min jumps = 2

Code

```
import java.util.*;

class Solution {
    // Function to compute the minimum number of jumps to reach the last index
    public int jump(int[] nums) {
        int n = nums.length;
        int[] dp = new int[n];

        // Fill dp array with max value
        Arrays.fill(dp, Integer.MAX_VALUE);

        // Start index requires 0 jumps
        dp[0] = 0;

        // Traverse all indices
        for (int i = 0; i < n; i++) {
            for (int j = 1; j <= nums[i] && i + j < n; j++) {
                dp[i + j] = Math.min(dp[i + j], dp[i] + 1);
            }
        }

        // Return min jumps to reach end
        return dp[n - 1];
    }
}

// Driver Code
public class Main {
    public static void main(String[] args) {
        Solution sol = new Solution();
        int[] nums = {2, 3, 1, 1, 4};
        System.out.println("Minimum jumps: " + sol.jump(nums));
    }
}
```

Complexity Analysis

Time Complexity: $O(N^2)$, We use two nested loops where outer loop runs for N elements and inner can go up to N in worst case.

Space Complexity: $O(N)$, We use an extra DP array of size N to store the minimum jumps to reach each index.

Optimal Approach

Algorithm

We need to reach the last index with the minimum number of jumps. Instead of trying all possible paths (as in brute force), we use a greedy approach. At every step, we try to go as far as we can within the current jump range. We treat each jump as moving through levels in a graph, updating the range of indices we can reach in the next jump.

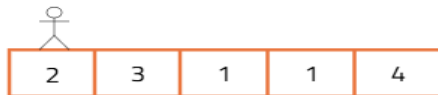
- Start from the first index in the array.
- Keep track of the furthest index you can reach so far in the current jump range.
- Iterate through each index up to the second-last element.
- For every index, update the maximum reachable index from this point.
- When you reach the end of the current jump range:
 - Increase your jump count.
 - Update the jump range to the furthest point you could reach so far.

- Continue until the loop completes.
- Return the total number of jumps needed to reach the last index.

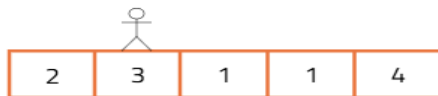
Dry run :



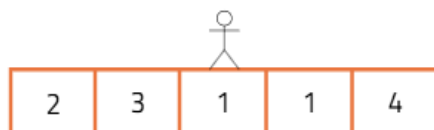
farthest = 0 currentend = 0 jumps = 0



farthest = max(0, 0 + 2) = 2 currentend = 2 jumps = 1



farthest = max(2, 1 + 2) = 4 i != currentend no jumps



farthest = max(4, 2 + 1) = 4 currentend = farthest = 4 jumps = 2

i == currentend



farthest = max(4, 3 + 1) = 4 i != currentend no jumps



reachedend, stop

min jump needed = 2

Code

```
import java.util.*;
```

```
class Solution {
    // Function to compute the minimum number of jumps to reach the last index
    public int jump(int[] nums) {
        // Initialize jumps and range trackers
        int jumps = 0;
        int currentEnd = 0;
        int farthest = 0;
    }
}
```

```

// Loop through array up to second last index
for (int i = 0; i < nums.length - 1; i++) {
    // Update the farthest index we can reach
    farthest = Math.max(farthest, i + nums[i]);

    // If current index reaches the end of current range
    if (i == currentEnd) {
        // Increment jump count
        jumps++;

        // Update range to the farthest index
        currentEnd = farthest;
    }
}

// Return the total number of jumps
return jumps;
}

// Driver Code
public class Main {
    public static void main(String[] args) {
        Solution sol = new Solution();
        int[] nums = {2, 3, 1, 1, 4};

        System.out.println("Minimum jumps required: " + sol.jump(nums));
    }
}

```

Complexity Analysis

Time Complexity: $O(N)$, We traverse the entire array once from start to end.

Space Complexity: $O(1)$, No extra space is used apart from a few integer variables.